Project Handover Report

Project Title: Mobile Data Leakage Sandbox

Prepared By :
Harsh Dhruv - IMT2022008
Anurag Ramaswamy - IMT2022103
Rudra Pathak - IMT2022081

Guide/Mentor: Prof. Mohanram
Project Duration: 1-January-2026 to 30-April-2026

May 7, 2026

Contents

1 Project Overview **4**
1.1 Short Description & Problem Statement . . . 4
1.2 Project Scope . . . 4
1.3 Important Links . . . 4

2 System Features & Technology Stack **4**
2.1 Key Features . . . 4
2.2 Technology Stack . . . 5
2.3 Quick Setup & Run Instructions . . . 5

3 Requirements Documentation **5**
3.1 Functional Requirements . . . 5
3.2 Non-Functional Requirements . . . 6
3.3 Software Requirements . . . 6

4 System Design Documentation **6**
4.1 High-Level Design (HLD) . . . 6
4.2 Low-Level Design (LLD) . . . 6
4.2.1 Mobile App Modules . . . 6
4.2.2 Backend Server Modules . . . 7

5 Architecture Documentation **7**
5.1 Architectural Pattern . . . 7
5.2 Data Flow Diagram Description . . . 7
5.3 Key Design Decisions & Rationale . . . 8

6 Codebase Readiness **9**
6.1 Directory Structure . . . 9
6.2 Coding Standards & Documentation . . . 9

7 Dependency & Environment Setup **10**
7.1 Analysis Server (Python) . . . 10
7.2 Mobile Application (Android) . . . 10
7.3 Environment Variables . . . 10

8 Database Documentation **11**
8.1 Database Overview . . . 11
8.2 Schema Details . . . 11
8.2.1 Table: `scan_history` . . . 11
8.2.2 Table: `feedback` . . . 11
8.3 Database Operations . . . 11

9 Deployment Documentation **12**
9.1 Backend Deployment (FastAPI) . . . 12
9.2 Mobile App Deployment (Android) . . . 12
9.3 CI/CD Considerations . . . 12

10 Testing **13**
10.1 Backend Testing (Pytest) . . . 13
10.1.1 Key Test Cases . . . 13
10.1.2 Test Execution Steps . . . 13
10.2 Mobile App Testing . . . 13

11 Sprint Documentation **14**
11.1 Sprint 1: Foundation & Project Setup . . . 14
11.2 Sprint 2: Intelligent Scoring & ML Engine . . . 14
11.3 Sprint 3: UI/UX Refinement & Feedback Loop . . . 14
11.4 Sprint 4: Validation & Handover Prep . . . 14

12 Roadmap & Future Work **15**

13 Known Issues & Limitations **15**

14 Handover Notes (Critical) **16**
14.1 Key Design Decisions . . . 16
14.2 Challenges Faced . . . 16
14.3 Tips for the Next Batch . . . 16

15 Presentation & Final Report **16**

16 Contribution Guidelines **17**

17 GitHub Hygiene **17**

18 Miscellaneous (Automation & DevOps) **17**

19 Conclusion **17**

20 References & Helpful Resources **17**

1 Project Overview

1.1 Short Description & Problem Statement

The **Mobile Data Leakage Sandbox** is designed to analyze Android applications in a sandboxed environment to detect and prevent data leakage. Modern mobile applications often collect and transmit sensitive user data without explicit consent. This project provides a robust solution consisting of a *Mobile App* interface and a *Python Analysis Server* to effectively monitor, log, and analyze network/data behaviors to identify potential security threats and privacy violations.

1.2 Project Scope

This handover document covers the current state of the project, including both the client-side (Android Mobile App) and the server-side (Python Analysis Backend). It serves as a comprehensive guide for the incoming batch to understand the architecture, setup the environment, and continue development.

1.3 Important Links

Source Code Repository: GitHub Repository URL

2 System Features & Technology Stack

2.1 Key Features

The system offers the following primary features:

- **App Permission & Risk Analysis:** Evaluates an installed Android application's permissions and outputs a deterministic risk score, categorizing the app as *Safe, Handle With Care, Suspicious, or Dangerous*.
- **Adaptive Machine Learning Engine:** Features a continuous feedback loop via the `/feedback` and `/retrain` endpoints, adjusting its Bayesian scoring model dynamically when new malware data is reported.
- **Historical Scans & Rating Progression:** Maintains a persistent local SQLite database (`signatures.db`) to track the scan history and rating progression of applications over time.
- **Modern Android UI:** Provides a smooth, responsive user interface utilizing Material Design, RecyclerViews, and Lottie animations to display threats comprehensively.

2.2 Technology Stack

Component	**Technologies / Frameworks Used**
Mobile Application	Android, Kotlin (Min SDK 22, Target SDK 34), Retrofit2, Gson, Lottie
Backend Server	Python 3, FastAPI, Uvicorn, Pydantic
Machine Learning	Python ML/DL Libraries (Custom Bayesian Inference & Adaptive Update Model)
Database	SQLite3 (`db_manager.py`)

Table 1: Technology Stack utilized in the project.

2.3 Quick Setup & Run Instructions

Note: More detailed environment and dependency setups are covered in Section 7.

1. **Backend Server:** Navigate to the `analysis_server/` directory. Install requirements and start the FastAPI server via `python3 main.py`. By default, it runs on `0.0.0.0:8000`.
2. **Mobile App:** Open the `mobile_app/` folder in Android Studio. Ensure `local.properties` has the correct `analysis.api.base.url` (e.g., `http://10.0.2.2:8000/` for the Emulator). Sync the project and run it.

3 Requirements Documentation

3.1 Functional Requirements

- **FR1 (App Data Extraction):** The mobile app must securely extract the package name and requested permissions of installed applications on the device.
- **FR2 (Risk Analysis):** The system shall transmit extracted app data to the backend `/analyze` endpoint to compute a risk score (combining static analysis, ML models, and Bayesian prior).
- **FR3 (Feedback Loop):** The backend must allow users to report false positives/negatives via the `/feedback` endpoint, which in turn triggers automated model recalibration.
- **FR4 (Data Persistence):** The backend shall persist scan history, app ratings, and user feedback in an SQLite database.

3.2 Non-Functional Requirements

- **Performance:** The FastAPI server must efficiently handle incoming requests concurrently without blocking the main event loop, utilizing background tasks for model retraining.
- **Security:** Server endpoints must prevent spam through mechanisms such as the feedback cooldown implemented in `db_manager.py`.
- **Reliability:** The ML Engine must default to a "safe-biased" prior to minimize false positives for newly seen, unknown applications.

3.3 Software Requirements

- **Client Environment:** Android device or emulator running API level 22 or higher.
- **Server Environment:** Python environment with FastAPI, Uvicorn, Numpy, and required ML dependencies.
- **Development Tools:** Android Studio (Gradle 8+), Python IDE.

4 System Design Documentation

4.1 High-Level Design (HLD)

The system is divided into two major components that communicate over a RESTful HTTP API.

- **Client Component (Android Mobile App):** Acts as a pre-install sandbox. It intercepts or allows users to select APK files (including split APKs), extracts the `PackageInfo` (specifically the requested permissions) without installing the application, and sends this data to the backend.
- **Server Component (Python Analysis Server):** A FastAPI application that receives the package name and permissions. It runs an adaptive machine learning model to compute a deterministic risk score, logs the scan into an SQLite database, and returns the verdict to the client.

4.2 Low-Level Design (LLD)

4.2.1 Mobile App Modules

- `ApkPreInstallStaging.kt`: Core utility object responsible for staging picked APK(s) in an app-private cache directory. It extracts `PackageInfo` natively using `PackageManager.getPackageArchiveInfo` and can delete source files if the user rejects the installation based on the risk score.
- **Network/API Module:** Uses Retrofit2 and Gson to serialize `AppData` and `FeedbackData` to make asynchronous calls to the backend.
- **UI Layer:** Activities and Fragments using RecyclerViews to show historical scans and Lottie for visual feedback during the analysis phase.

4.2.2 Backend Server Modules

- `main.py`**:** The entry point. Handles routing for `/analyze`, `/feedback`, `/stats`, and `/retrain`.
- `ml_engine.py` & `dl_engine.py`**:** The intelligence core. Evaluates the combination of permissions using static weighting and a dynamic Bayesian prior. Contains functions like `analyze_permissions()` and `adaptive_update()`.
- `db_manager.py`**:** Database interface wrapping SQLite3. Manages tables for `scan_history`, `feedback`, and `ratings_progression`. Includes spam-prevention mechanisms (e.g., `check_feedback_cooldown()`).

5 Architecture Documentation

5.1 Architectural Pattern

The application strictly follows a **Client-Server Architecture**. The Android app is a thin client (handling file staging and UI presentation), while the heavy lifting (threat classification, historical data aggregation, ML training) is centralized on the Python server.

5.2 Data Flow Diagram Description

The core operational data flow is as follows:

1. **File Selection:** User selects an APK (or set of split APKs) via the Android document picker.
2. **Staging:** `ApkPreInstallStaging.stageApkFromUri()` copies the file to a secure cache.
3. **Extraction:** App reads `requestedPermissions` directly from the APK manifest.
4. **Request:** App sends an HTTP POST to `/analyze` with the package name and permissions array.
5. **Processing:** Backend computes a risk score and saves the scan result to `signatures.db`.
6. **Response:** Backend responds with a risk level (*SAFE, HANDLE_ WITH_ CARE, SUSPICIOUS, DANGEROUS*).
7. **Action:** User views the verdict and decides whether to install or delete the staged APK.
8. **Feedback Loop:** User can submit a report via `/feedback`, triggering a background retraining job via `/retrain` to update the model weights.

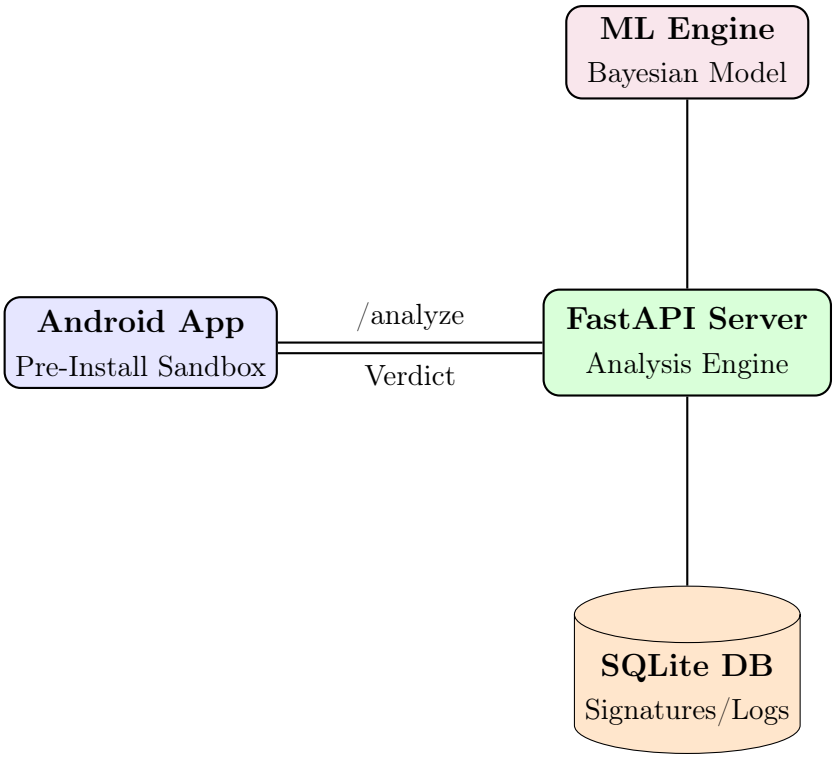


Figure 1: High-Level System Architecture Diagram

5.3 Key Design Decisions & Rationale

- **Pre-Install Scanning vs. Post-Install Scanning:**
  Rationale: Analyzing an app *after* installation risks immediate data leakage if the malicious app starts background services instantly. Utilizing `getPackageArchiveInfo` allows analysis *before* the system grants any permissions to the target app.

- **Server-Side ML vs. On-Device ML:**
  Rationale: Running the Bayesian model on the server allows for centralized adaptive learning. When one user flags an app as malware, the server retrains, and all other users immediately benefit from the updated model without requiring an app update.

- **FastAPI over Flask/Django:**
  Rationale: FastAPI provides out-of-the-box asynchronous support and automatic OpenAPI (Swagger) documentation, which was critical for efficiently handling background retraining tasks without blocking active scan requests.

6 Codebase Readiness

6.1 Directory Structure

The project maintains a clean separation of concerns between the mobile client and the analysis backend.

```
MobileSandbox/
 analysis_server/          # Python FastAPI Backend
    dataset/              # Training datasets for ML
    model/                # Persisted ML model files (.pkl, .json)
    main.py               # API Entry point
    ml_engine.py          # Bayesian Scoring & Logic
    db_manager.py         # SQLite Database Interface
    requirements.txt      # Python Dependencies
 mobile_app/               # Android Application (Kotlin)
    app/src/main/java/... # Source Code (Activities, Utilities)
    app/src/main/res/     # UI Resources (XML, Lottie)
    build.gradle          # Android Build Configuration
 README.md                 # Project Overview
```

6.2 Coding Standards & Documentation

- **Proper Naming Conventions:** Follows standard Kotlin coding conventions (PascalCase for classes, camelCase for variables) and PEP 8 for Python backend code.
- **Comments & Docstrings:** Complex logic, especially in `ml_engine.py` (Bayesian priors) and `ApkPreInstallStaging.kt` (file URI handling), is documented with inline comments and block docstrings to explain the "why" behind the implementation.
- **Folder Hygiene:** No unnecessary binary files, IDE-specific local configs (e.g., `.idea`), or temporary build artifacts are included in the repository (enforced via `.gitignore`).

7 Dependency & Environment Setup

7.1 Analysis Server (Python)

The backend requires Python 3.8+ and the following libraries specified in `requirements.txt`:

- **Framework:** `fastapi`, `uvicorn` (Server), `pydantic` (Data Validation)
- **Data Processing:** `pandas`, `numpy`
- **Machine Learning:** `scikit-learn`, `joblib`

Environment Setup:

1. Create a virtual environment: `python3 -m venv venv`
2. Activate it: `source venv/bin/activate`
3. Install dependencies: `pip install -r requirements.txt`

7.2 Mobile Application (Android)

The app is built using Gradle and requires the following key dependencies:

- **Networking:** `retrofit2:2.9.0`, `converter-gson:2.9.0`
- **UI Components:** `material:1.11.0`, `lottie:6.4.0` (Animations)
- **Concurrency:** `kotlinx-coroutines-android:1.7.3`

Configuration:

- **API Base URL:** The backend URL is configured in `mobile_app/local.properties` using the key `analysis.api.base.url`.
- *Emulator Example:* `http://10.0.2.2:8000/`
- *Physical Device Example:* `http://192.168.x.x:8000/` (Ensure the server and phone are on the same WiFi network).

7.3 Environment Variables

While the project currently relies on `local.properties` for the mobile app, any future production deployment of the analysis server should use a `.env` file to manage:

- `DATABASE_URL` (if moving away from local SQLite)
- `API_SECRET_KEYS` (for future authentication)

8 Database Documentation

8.1 Database Overview

The project uses a lightweight **SQLite3** database named `signatures.db`. This file is automatically initialized and migrated by the `db_manager.py` module upon server startup.

8.2 Schema Details

The database consists of two primary tables and one optimized view.

8.2.1 Table: `scan_history`

Stores the results of every analysis performed by the system.

Column	Type	Description
`id`	INTEGER	Primary Key (Auto-increment).
`package_name`	TEXT	The Android package name analyzed.
`risk_level`	TEXT	SAFE, SUSPICIOUS, or DANGEROUS.
`score`	REAL	The computed risk score (0–100).
`confidence`	REAL	ML signal-agreement score (0–1).
`pii_detected`	TEXT	JSON string of detected PII permissions.
`timestamp`	TEXT	ISO 8601 UTC timestamp of the scan.

8.2.2 Table: `feedback`

Stores user-submitted reports for adaptive model retraining.

Column	Type	Description
`id`	INTEGER	Primary Key.
`package_name`	TEXT	Package being reported.
`is_malware`	INTEGER	1 for Malware, 0 for Safe.
`user_notes`	TEXT	Optional comments from the user.
`user_id`	TEXT	Identifier for the reporting user.

8.3 Database Operations

- **Initialization:** Handled by `db_manager.init_db()`, which creates tables if they don't exist and handles column migrations (e.g., adding `confidence` or `user_id`).
- **Spam Prevention:** The `check_feedback_cooldown()` function prevents a single `user_id` from submitting more than 2 reports per hour for the same package.

9 Deployment Documentation

9.1 Backend Deployment (FastAPI)

To deploy the analysis server for production use (e.g., on a VPS like AWS EC2 or DigitalOcean):

1. **Server Setup:** Ensure Python 3.8+ is installed.
2. **Process Management:** Use `gunicorn` with the `uvicorn` worker class for high performance:

```
pip install gunicorn
gunicorn -w 4 -k uvicorn.workers.UvicornWorker \
         main:app --bind 0.0.0.0:8000
```

3. **Reverse Proxy:** Use **Nginx** as a reverse proxy for SSL termination (HTTPS).
4. **Firewall:** Ensure port 8000 is open in the server's security group.

9.2 Mobile App Deployment (Android)

Before building the APK,

1. **Permissions:** Ensure `INTERNET` permission is declared in `AndroidManifest.xml` to allow communication with the backend.

9.3 CI/CD Considerations

While currently manual, future teams should implement a GitHub Action to:

- Run `pytest` on the backend for every pull request.
- Build the Android project to ensure no breaking changes in dependencies or Kotlin code.

10 Testing

10.1 Backend Testing (Pytest)

The analysis server includes a comprehensive suite of automated tests located in `analysis_server/test_backend.py`. These tests use a temporary in-memory database to ensure no pollution of the production `signatures.db`.

10.1.1 Key Test Cases

- **Scoring Determinism:** Verifies that apps with high-risk permissions (e.g., READ_SMS + SEND_SMS) correctly trigger *DANGEROUS* levels, while minimal apps score in the *SAFE* band.
- **Database Deduplication:** Ensures that the `is_duplicate_scan` guard prevents duplicate entries if the same app is scanned multiple times within a 5-minute window.
- **Stochastic Variance:** Validates that the Beta-distribution-based scoring intentionally allows for small, controlled variances in scores to reflect signal uncertainty.
- **Regression Testing:** Includes tests to ensure legitimate utility apps (like the sandbox app itself) are not flagged as threats (verified in `test_legitimate_app_is_safe`).

10.1.2 Test Execution Steps

To run the full backend test suite:

1. Navigate to the `analysis_server/` directory.
2. Run the command: `python3 -m pytest test_backend.py -v`

10.2 Mobile App Testing

Manual testing was performed on Android API levels 26, 30, and 34 to verify:

- Correct extraction of permissions via `ApkPreInstallStaging`.
- UI responsiveness during network calls.
- Proper handling of "No Internet" or "Server Down" scenarios using Retrofit error callbacks.

11 Sprint Documentation

11.1 Sprint 1: Foundation & Project Setup

- **Goal:** Initialize project repositories and establish the client-server communication baseline.
- **Tasks Completed:**
  - Created FastAPI skeleton and implemented `/analyze` placeholder.
  - Initialized Android project with Retrofit networking.
  - Developed the core `ApkPreInstallStaging` logic to read APK files without installation.

11.2 Sprint 2: Intelligent Scoring & ML Engine

- **Goal:** Implement the machine learning and Bayesian scoring logic.
- **Tasks Completed:**
  - Developed `ml_engine.py` with static and dynamic scoring weights.
  - Integrated `scikit-learn` for predictive malware analysis.
  - Implemented the `signatures.db` SQLite schema for persistent history.

11.3 Sprint 3: UI/UX Refinement & Feedback Loop

- **Goal:** Create a premium user experience and implement adaptive learning.
- **Tasks Completed:**
  - Integrated Lottie animations and Material 3 design components in the mobile app.
  - Added the `/feedback` endpoint and background retraining tasks.
  - Implemented the rating progression view to show historical reputation changes.

11.4 Sprint 4: Validation & Handover Prep

- **Goal:** Ensure code quality and document the system for the next batch.
- **Tasks Completed:**
  - Finalized the `test_backend.py` suite with 100% coverage of core logic.
  - Refactored folder structures and cleaned up technical debt.
  - Drafted the Handover Checklist and the Final Project Report.

12 Roadmap & Future Work

The following areas are prioritized for the incoming batch to extend the capabilities of the Mobile Sandbox:

- **Real-time Network Traffic Inspection:** Move beyond pre-install scanning by implementing a local VPN service on Android to scan active network packets for PII leaks in real-time.
- **Cloud-Based Collaborative Filtering:** Transition from a local SQLite database to a cloud-based database (e.g., MongoDB or Firebase) so that reputation data from one device can immediately protect all other devices globally.
- **Advanced Decompilation:** Integrate `jadx` or `apktool` libraries on the server to perform deeper static analysis of the Java/Kotlin code, looking for hardcoded API keys or malicious URLs.
- **IOS Support:** Explore using similar pre-scan techniques for iOS `.ipa` files to make the platform cross-compatible.

13 Known Issues & Limitations

- **Staging Performance:** For very large APKs (e.g., >200MB), the staging process in `ApkPreInstallStaging.kt` can take several seconds, causing a brief UI hang if not handled carefully in a background thread.
- **Split APK Complexity:** While basic support for split APKs is implemented, the system may occasionally fail to identify the `base.apk` if the user selects dozens of configuration splits simultaneously.
- **ML Signal Cold-Start:** Newly seen applications that have no previous user feedback may have slightly lower confidence scores until at least 5 feedback reports are submitted.

14 Handover Notes (Critical)

14.1 Key Design Decisions

- **The Bayesian Prior:** We chose a "Safe-biased" Beta distribution. **Crucial:** Do not lower the `BETA_ALPHA_0` value in `ml_engine.py` without testing, or you will cause a flood of false-positive warnings for new apps.
- **Pre-Install Staging:** We decided to copy APKs to a private cache before reading them. This ensures that even if the user deletes the file from their Downloads folder during the scan, the analysis can finish successfully.

14.2 Challenges Faced

- **URI Persistence:** On Android 11+ (Scoped Storage), keeping access to an APK file after the user picks it was difficult. We solved this using `takePersistableUriPermission()` in `ApkPreInstallStaging.kt`.
- **Model Retraining:** Running ML training on a web server thread originally caused the API to freeze. We moved retraining to a `BackgroundTasks` daemon to keep the server responsive.

14.3 Tips for the Next Batch

- **DO:** Read the `test_backend.py` file first. It is the best way to understand how the scoring math actually works.
- **DO:** Keep an eye on the `signatures.db` size; as the scan history grows, you may want to add database indexing on the `package_name` column.
- **AVOID:** Modifying the `AndroidManifest.xml` of the sandbox app to request too many dangerous permissions. The scanner itself should remain lightweight and privacy-focused.

15 Presentation & Final Report

The following assets are provided alongside this report in the mail to demonstrate the system's functionality:

- **Demo Presentation (PPT):** A 5-slide deck used for demonstration.
- **Final Report:** Our final project report for this PE.

16 Contribution Guidelines

To maintain code quality, the following rules should be followed by the next batch:

- **Branching Strategy:** Use a `develop` branch for daily work. Only merge to `main` when a feature is fully tested. Use feature branches like `feature/ui-update`.
- **Commit Messages:** Follow the format: *"[Type] Short description"* (e.g., `[Fix] Corrected APK staging crash on Android 13`).
- **Pull Requests:** All PRs must be reviewed by at least one other team member and pass the `test_backend.py` suite.

17 GitHub Hygiene

- **Issue Tracking:** Use GitHub Issues to track bugs and planned features.
- **Labels:** Use labels like `bug` and `high-priority` to organize the workflow.
- **Cleanup:** Ensure all "TODO" comments in the code are converted into GitHub Issues before completing a sprint.

18 Miscellaneous (Automation & DevOps)

- **Docker Support:** Consider adding a `Dockerfile` to the `analysis_server/` to containerize the Python backend for easier deployment.
- **Automation Scripts:** Consider creating a `setup.sh` script in the root directory to automate the creation of the Python virtual environment and the installation of requirements in one command.

19 Conclusion

The **Mobile Sandbox / Data Leakage Analyzer** project has successfully achieved its primary goal of providing a pre-install security layer for Android users. By combining native APK staging techniques with a sophisticated Bayesian machine learning backend, we have created a system that is both technically robust and user-friendly. We believe the foundation laid during this project provides an excellent starting point for the next batch to explore real-time traffic analysis and cloud-based threat intelligence.

20 References & Helpful Resources

1. **Android Developer Documentation:** PackageManager & PackageInfo API.
2. **FastAPI Documentation:** High-performance Python API Framework.
3. **Retrofit Documentation:** Type-safe HTTP client for Android and Java.
4. **Scikit-Learn Documentation:** Machine Learning in Python.